C^3: Committed Confidence Calibration

Detailed research plan: what we do, how we do it, and what we expect

Abhiraj Ratna · Ayush SM · Ojas Kataria · Parth Sharma Version 2 · 25 September 2026

Goal. Publish this work as an 8-page main-track paper at an A* venue. **Primary target: ICML 2027** (deadline expected late January 2027). **Fallbacks:** ACL Rolling Review (February 2027 cycle) and NeurIPS 2027 (around May). Check the real dates as soon as they are announced. **How to read this document.** Section 1 explains the project in plain words. Section 2 explains what changed from the proposal and why. Section 3 describes the setup that every experiment shares. Section 4 is the core: fifteen tasks, each with *why*, *what we run*, *how, step by step*, *what we expect*, *when it is done*, and *what to do if it goes wrong*. Sections 5–10 cover statistics rules, the timeline, the team, the paper outline, risks, and the final checklist.

1 The project in plain words

How a diffusion language model writes text. A diffusion LM such as LLaDA-8B or Dream-7B starts with a sequence of blank (masked) slots. In each step it looks at the whole sequence once (one *forward pass*) and predicts a probability distribution over words for every blank slot. The *decoder* then chooses a few slots to fill in for good, which we call *committing* them, and repeats until no blanks are left. Nearly every decoder chooses the slots whose top prediction has the highest probability. That probability is the slot's *confidence*.

The worry. Confidence is an estimate, and estimates have errors. When you pick the highest of many noisy estimates, you tend to pick one whose error happened to push it up. Think of 32 students each guessing their own exam score: the one with the highest guess has probably also overestimated the most. Economists call this the *winner's curse*. For a decoder, it would mean the confidence of a committed token overstates how likely that token is to be right, and more so when the token beat a larger field of candidates (early steps, long outputs).

What the paper will show. We will measure four things and build one fix:

1. **E1, the committed-token gap.** How far a committed token's confidence exceeds its real accuracy, as a function of how many candidates it beat. We will also show that the standard calibration score (ECE) underestimates this gap.
2. **E2, the field effect.** Whether winning a large field adds overconfidence that the token's own confidence does not reveal. This is the real winner's curse, and Section 2 explains why the proposal's original test could not detect it.
3. **E3, trajectory shift.** Whether the model is less calibrated in the partly filled states that a decoder actually visits than in the random states it was trained on.
4. **C2, two sources of error.** Parallel decoding goes wrong for two separate reasons, and we measure each in exact units (nats) on a synthetic dataset where the truth is known. First, the model does not know how tokens depend on each other (the *dependency* term). Second, each token's own probability is off (the *estimation* term).
5. **C3, a free fix.** A recalibration step that uses only numbers the model already computed, so it costs no extra forward passes. We test whether it gives better accuracy at the same decoding speed.

Symbols used throughout.

x_0	the true document (or true answer)	v_i	the model's top-1 token at masked position i
c_i	confidence: probability of v_i	$\ell_i = \log \frac{c_i}{1-c_i}$	confidence as log-odds, so $c_i = \sigma(\ell_i)$
$a_i = \mathbb{1}\{v_i = x_{0i}\}$	correct (1) or wrong (0)	s	a *state*: one partly masked sequence, one forward pass
M	the masked positions in a state	m	*field size*: how many candidates were compared
r	rank of a position within its pool	$\pi(\ell)$	$\Pr(a = 1 \mid \ell)$, the ordinary calibration curve
$\Delta(m)$	mean of $c_i - a_i$ over tokens that won a pool of size m	(A), (B)	dependency and estimation terms of Eq. (1)

2 What changed from the proposal, and why

We tested the proposal's key argument (§5.1–5.2) in simulation before spending any compute. It needs one correction. The correction does not kill the idea. It shows where the effect actually lives and which test can see it.

Proposition 1 (Winning adds nothing once you know the confidence *and* the context)**.** *Fix a state s. Suppose that, given s, the pairs (ℓ_j, a_j) are independent across masked positions, and that the pool P is drawn at random. Then for the*

position i that wins the pool,

$$\Pr\left(a_i = 1 \,\middle|\, \ell_i,\ s,\ i = \arg\max_{j \in P} \ell_j\right) \;=\; \Pr\left(a_i = 1 \,\middle|\, \ell_i,\ s\right) \;=:\; \pi(\ell_i, s).$$

Why this holds: once ℓ_i and s are known, whether i wins depends only on the other positions' scores, and those carry no information about a_i. This is Efron's (2011) point that selection does not bias a correct posterior. It holds even if some positions are much noisier than others, so the proposal's claim that unequal noise alone makes field size informative is not right.

Proposition 2 (Exact split of the gap). *Let $\mathbb{E}_m$ denote an average over tokens that win pools of size m. Under the conditions of Proposition 1,*

$$\Delta(m) \;=\; \underbrace{\mathbb{E}_m\left[\sigma(\ell) - \pi(\ell)\right]}_{\text{ordinary miscalibration, seen through selection}} \;+\; \underbrace{\mathbb{E}_m\left[\pi(\ell) - \pi(\ell, s)\right]}_{\text{field term}}.$$

The first term is what a standard calibration curve predicts, re-weighted toward where winners sit. The second term, the *field term*, is nonzero only when two things are both true. First, accuracy at a fixed confidence depends on the context: a score of 0.9 in a hard document is less trustworthy than in an easy one. Second, winning a big pool changes which contexts you end up in: a lone high score in a weak field is more likely to be noise. At $m = 1$ the field term is exactly zero. **The field term is the real winner's curse.**

Why the proposal's regression (Eq. 2) could not see it. Eq. 2 includes *state fixed effects*, which are a separate intercept for every state. The field term lives entirely in differences between states, so the fixed effects remove it. In our simulation, which uses heteroskedastic noise and states of varying difficulty, $\hat{\beta}_z = 0.03$ (s.e. 0.03) *with* state fixed effects and $\hat{\beta}_z = -0.29$ (s.e. 0.017) *without* them. The second value has exactly the sign the proposal predicted. There is also a second issue. At a fixed state, each position's ℓ_i and a_i are fixed numbers, so drawing many random pools only re-weights positions we already have. It adds no new data. Sample size must count documents and positions, not pool draws.

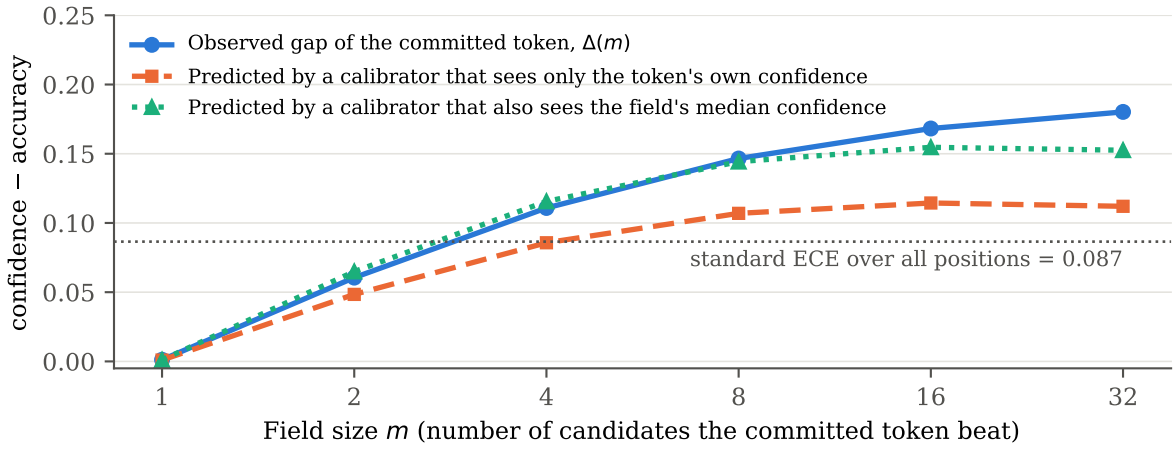


Figure 1. What E1 and E2 look like in our simulation (the `sim_field.py` script). The committed token's gap grows from 0.1% to 18% as m grows. The standard ECE (8.7%) misses this. A calibrator that uses only the token's own confidence predicts 11% at $m = 32$. The remaining 7% is the field term, and a calibrator that also sees the field's median confidence recovers most of it.

This updates the two-page plan. That plan said the effect lives only in the unconditional gap. The more careful simulation shows that part of it is the field term, which is the proposal's original hypothesis in a corrected, testable form. So E2 is now a main result rather than a side check, and C3 becomes a context-aware recalibration.

In the proposal	In this plan
Primary endpoint: β_z in Eq. 2 with state fixed effects	Primary: $\Delta(m)$ (E1) and the field term (E2). Eq. 2 is fitted *with and without* fixed effects, and the pair of results tells us where the effect lives.
Randomising pools gives identification	Pools are only a way to compute averages; documents are the unit of analysis.
C3 only if C1 succeeds	C3 always runs: a context-aware recalibration using no extra forward passes.
JSON probe with independent fields	Probe gets a dependency knob ρ, so the dependency term (A) varies from zero upward.
8B models run in 4-bit	8B models run in bfloat16; 4-bit becomes an ablation.

3 The setup every experiment shares

3.1 Models

Model	Used for	Why it is in the paper
110M MDLM (trained by us, T3)	everything	The only model whose training data we control, so "held-out" is guaranteed. Cheap enough for exact computations. Base model for the JSON probe.
LLaDA-8B Base / Instruct	E1–E3, C3, tasks	The most-used open diffusion LM. Use Base on raw web text and Instruct on GSM8K and MBPP.
Dream-7B Base / Instruct	E1–E3, C3, tasks	A second 8B-scale family with a different training recipe, to show results are not specific to one model.
Third model (W11)	E1, E2 only	For example LLaDA-1.5 or a block-diffusion model. This answers "does it generalise?".

3.2 Data

Dataset	Size	Role
FineWeb-Edu 10BT (held-out)	2,000 docs	Main calibration corpus. Each document is truncated to its first 256 tokens, and documents are never packed together.
Time-split set (new, T4)	500 docs	Text written after the 8B models were trained, so it is guaranteed unseen. Robustness check.
GSM8K test / MBPP test	1,319 / 500	Question as the prompt, reference answer as the masked target, for teacher-forced measurements. Also used as downstream tasks.
HumanEval	164	Downstream pass@1 only (too small for fitting).
Synthetic JSON probe (T9)	1,000 eval + 200k train	The true distribution is known, so C2 can be computed exactly.

3.3 The basic measurement: teacher-forced infilling

Every measurement is built from the same step. Take a true document x_0 and hide (mask) some positions. Run *one* forward pass. For every masked position i, read off the top-1 token v_i, its confidence c_i, and whether it is right: $a_i = \mathbb{1}\{v_i = x_{0i}\}$. The model never fills in its own guesses, so correctness is measured against the truth with no knock-on errors from earlier mistakes.

We create masked states in two ways:

- **Uniform masks**, the way the model was trained. Pick a mask rate $t \in \{0.1, 0.2, \ldots, 0.9\}$ and hide a random fraction t of positions. Use 2 random masks per rate per document.
- **Decoder-induced masks**, the states a decoder actually visits. Start fully masked and let a decoder (for example, top-k by confidence) choose which positions to commit at each step. Then fill those positions with the *true* tokens, not the model's guesses, and continue. Record every state along the way. This shows exactly which states a decoder would visit if it were always right. Run it for $k \in \{2, 4, 8\}$ tokens per step and for each decoder in T12.

3.4 What we save (the trace), and how much compute it costs

One row per masked position per state, stored in Parquet files: document ID, state ID, mask type, mask rate, step, number of masked positions $|M|$, position index, v_i, c_i, ℓ_i, probability of the true token, entropy, margin (top-1 minus top-2), the top-20 token IDs and probabilities, the change in ℓ_i since the previous step, a_i, whether the position was committed and at what rank, model, precision, seed, and git commit. We do **not** store full probability vectors: LLaDA's vocabulary of about 126k tokens × 256 positions × 2 bytes is about 64 MB per forward pass. A forward pass of an 8B model on 256 tokens takes a fraction of a second on one A100, so all traces for one model come to a few tens of GPU-hours and roughly 10 GB of storage.

Splits. Documents are split 50/50 with a fixed seed. Everything we *fit* (calibrators, splines, temperatures, C3) uses the first half. Every number we *report* comes from the second half.

4 The tasks in detail

Each box gives the owner, the weeks, and what the task depends on. The suggested owners are a starting point; reassign freely.

T0 Theory and simulations — Abhiraj · W1 (28 Sep–4 Oct)

Depends on: nothing. Everything else depends on it.

Why. Reviewers at A* venues expect the logic to be stated formally, and the simulations tell us in advance what each possible real result would mean.

What we run.

1. Write clean proofs of Propositions 1 and 2, each under half a page, for the appendix.
2. Try to prove that the field term grows with m in the simple Gaussian model used in Figure 1. If the proof is hard, show it numerically and call it an observation.
3. Extend `sim_field.py` to four scenarios and record the pattern of results each one produces:

Scenario (simulated truth)	$\Delta(m)$ grows?	field term	β_z, no FE	β_z, with FE
Model perfectly calibrated in every context	no	0	0	0
Miscalibrated, but the same way in every context	yes	0	0	0
Miscalibration depends on context (Figure 1)	yes	> 0	< 0	0
Errors shared across positions (breaks Prop. 1)	yes	?	< 0	$\neq 0$ (to confirm)

How. Reuse the attached scripts. Put all four scenarios in one file that prints this table, with seeds fixed. For the "shared errors" scenario, give each state a hidden "misreading" variable that shifts several positions' scores together.

What we expect. The first three rows as shown. The fourth row is the one we are least sure about, which is why it is simulated first.

Done when. A second teammate has checked the proofs line by line, and one command reproduces the table and Figure 1. This is **Gate G0**: all four of us and our advisor agree on the framing.

If it goes wrong. If someone finds a flaw, fix the framing before the pilot. The pilot measures raw quantities, so it can start anyway.

T1 Trace pipeline and model setup — Ayush · W1–W2

Depends on: GPU access. Everything experimental depends on it.

Why. Every experiment reads the same trace files. Building them once and carefully prevents a dozen subtle bugs later.

What we run. A script `trace.py -model -dataset -masks` that writes the Parquet trace of Section 3.4.

How.

1. Load the four 8B checkpoints in **bfloat16** with Hugging Face `transformers`. This needs about 16 GB of GPU memory, which fits on a 24–40 GB card.
2. Implement uniform masks and teacher-forced decoder trajectories (Section 3.3). For GSM8K and MBPP, apply the chat template to the question and mask only the answer.
3. Compute all per-position fields in one pass on the GPU, and move only the top-20 values to the CPU.
4. Fix every seed and turn on PyTorch's deterministic settings. Then run 20 documents 5 times and record the largest change in ℓ and in $\Delta(m)$. This is the *non-determinism floor*.
5. Write unit tests: the number of masked positions matches the mask; in teacher-forced mode no true token is visible before it is committed; a bfloat16 vs float32 spot check on 2 documents.

What we expect. Well under a second per forward pass, and a floor small compared with the effects we care about (well below one percentage point).

Done when. Traces for 20 documents are reproducible within the floor, all tests pass, and the schema is frozen (only additions allowed after W4).

If it goes wrong. If only small GPUs are available, run the main sweep on the 110M model and 8B only on a subsample. Never switch the main 8B runs to 4-bit.

T2 Pilot, power analysis, and pre-registration — Abhiraj + Ayush · W2 (5–11 Oct)

Depends on: T0, T1.

Why. To learn whether the effects are large enough to detect with our budget, and to fix our analysis choices before seeing the main data. Pre-registration is strong protection against the reviewer comment "you tuned the analysis until it worked".

What we run. 20 FineWeb documents (kept out of the main set) on LLaDA-8B-Base, with uniform masks at 5 rates and a top-k ($k = 4$) teacher-forced trajectory.

How. Compute $\Delta(m)$ for $m \in \{1, 2, 4, 8, 16, 32\}$, the ECE, the field term (T6 method), and the floor. For the key contrast $\delta = \Delta(32) - \Delta(1)$, compute it per document and take its standard deviation σ_d across documents. With a smallest effect worth reporting $\delta_{\min}$, fixed *before* looking (we suggest 2 percentage points), the number

of documents needed for 80% power at 5% significance is

$$N = \frac{(z_{0.975} + z_{0.80})^2 \, \sigma_d^2}{\delta_{\min}^2} \approx \frac{7.85 \, \sigma_d^2}{\delta_{\min}^2}.$$

Do the same for the field term. Then write the pre-registration: hypotheses, endpoints, analysis code, thresholds, and exclusion rules. Upload it to OSF with a timestamp before any evaluation-half trace is analysed.

What we expect. Honestly, we do not know the size. Base language models are often close to calibrated on their own training distribution, so under uniform masks the gap may be small (a few points). We expect bigger gaps for Instruct models and in decoder-induced states. The pilot tells us which.

Done when. **Gate G1**: if $N \leq 1{,}000$ documents, we run the full plan. If not, we use all 2,000 documents on fewer 8B settings, or shift weight to the 110M model and C2.

T3 Train our own 110M diffusion model — Ojas · W3–W4 (12–25 Oct), runs in the background

Depends on: the data split from T1.

Why. It is the only model where we *know* the evaluation documents were never seen, so claims about the *size* of calibration errors rest on it. It is also cheap enough for the exact probe computations.

How.

1. Use the public MDLM codebase (Sahoo et al., 2024): a 12-layer transformer with hidden size 768 and 12 heads, and the GPT-2 tokenizer.
2. Data: the FineWeb-Edu 10BT sample with our 2,000 evaluation documents, the pilot documents, and the time-split set removed. Remove them both by document ID and by a hash of their first 256 tokens.
3. Train at sequence length 256 without packing documents together, so training matches how we evaluate.
4. Budget: 10–20B tokens. Compute is about $6 \times (1.1 \times 10^8) \times (10^{10}) \approx 7 \times 10^{18}$ operations per 10B tokens, roughly one A100-day.
5. Sanity checks: a validation loss curve, the perplexity bound on held-out FineWeb, and readable samples. If budget allows, also train a same-size autoregressive GPT-2 on the same data as a reference.

What we expect. Perplexity somewhat worse than the autoregressive reference, which is normal for this model family, and a loss that has flattened by the end.

Done when. **Gate G2**: the loss is flat over the last 10% of training, and perplexity is in a sensible range compared with the MDLM paper and our autoregressive reference.

If it goes wrong. Fall back to the public MDLM checkpoint trained on OpenWebText, and run the contamination screen (T4) on it.

T4 Contamination screen and a guaranteed-unseen test set — Ayush · W3–W4

Depends on: T1.

Why. LLaDA and Dream were trained on web data that is not fully disclosed, so they may have seen some FineWeb-Edu documents. A memorised document looks "too accurate" and distorts calibration levels.

How. (1) For each document, give the model its first 50 tokens and let it generate the next 50 greedily. Measure the longest exact match with the true continuation. Fix the flag threshold *before* looking (we suggest 32 of 50 tokens matching). (2) Build a 500-document *time-split* set from text first published after the later of the two models' release dates, for example Wikipedia articles created after that date. Run the same trace pipeline on it.

What we expect. Only a small fraction of documents flagged. Levels of calibration may shift slightly when flagged documents are removed, but the *shape* of $\Delta(m)$ should not.

Done when. Every main result can be reported three ways: all documents, flagged documents removed, and the time-split set.

T5 E1: the committed-token gap (headline result) — Ayush (runs) + Abhiraj (analysis) · W5–W7 (26 Oct–15 Nov)

Depends on: T1, T2, T4 (T3 for the 110M arm).

Why. This is the paper's Figure 1. It shows that decoders operate where calibration is worse than standard metrics suggest.

What we run. $\Delta(m)$ for every model, dataset (FineWeb, GSM8K, MBPP), and mask type (uniform, decoder-induced).

How.

1. *Pool version.* In every recorded state, for each $m \leq |M|$, draw 20 random pools of size m with fixed seeds. Take the top position by ℓ and record $c_i - a_i$. Also record the top 4 of each pool, for rank analysis.
2. Average within each document first, then across documents, so long documents do not dominate. Compute

95% intervals with the document bootstrap (Section 5).
3. *Native version.* From the decoder-induced trajectories, take every token the decoder actually committed and group it by field size $|M_t|$ (the number of masked positions when it was chosen) and by step. This is the version that matters for real decoding.
4. Compare with ECE (15 equal-mass bins, plus smooth ECE) over all masked positions, and draw reliability diagrams.

What we expect. $\Delta(1)$ is close to the overall calibration error. $\Delta(m)$ rises with m if the model is overconfident at high confidence, with bigger gaps early in decoding and for Instruct models. If the curve is flat, that is also a publishable finding: it would explain *why* simple thresholding works as well as it does.
Done when. A figure of $\Delta(m)$ with intervals for all models against the ECE line, plus a native-version panel.

T6 E2: the field effect (the real winner's curse) — Abhiraj · W5–W7

Depends on: T5 traces.
Why. This separates "ordinary miscalibration seen through selection" from "winning a big field adds overconfidence you can't see in the token's own score". Only the second is the winner's curse, and only it motivates a context-aware fix.
How. Four analyses, all fitted on the first half and reported on the second:

1. **Prediction test.** Fit $\hat{\pi}(\ell)$ with isotonic regression on all masked positions. Predict $\hat{\Delta}_{\text{own}}(m) = \mathbb{E}_m[\sigma(\ell) - \hat{\pi}(\ell)]$ and compare it with the observed $\Delta(m)$. The difference estimates the field term (Proposition 2).
2. **Context-aware calibrator.** Fit a logistic model with a spline in ℓ plus *field features*: the median ℓ of the state's masked positions, the mask rate, $|M|$, and the step. If this closes the gap, as in Figure 1, the field term is real and C3 can fix it.
3. **Eq. 2 twice**, fitted on winning tokens only:

$$\Pr(a_i = 1) = \sigma\big(\alpha\ [+\,\alpha_{\text{state}}]\ +\ f(\ell_i)\ +\ \beta_z\, z_{r,m}\big),$$

   with f a cubic spline and $z_{r,m}$ the expected order statistic. We fit it once *without* and once *with* the state intercepts α_{state}. Do the same for the rank profile γ_k of Eq. 3. Read the result with the T0 table: $\beta_z < 0$ without and ≈ 0 with fixed effects means a field effect. $\beta_z \neq 0$ even with them means errors shared within a state.
4. **Direct check of shared errors.** The within-state correlation of residuals $a_i - \hat{\pi}(\ell_i, s)$ across positions (intra-class correlation).

What we expect. From the simulation, a positive field term that grows with m. In real models it could be small, because a language model reads the whole context and may already "know" a document is hard. A zero field term is still a clean result: it means the gap is fully explained by the ordinary calibration curve, and a simple per-step recalibration is enough.
Done when. A table of β_z (with and without fixed effects) and the ICC per model, and a plot of Δ, $\hat{\Delta}_{\text{own}}$ and $\hat{\Delta}_{\text{field}}$ like Figure 1.

T7 E3: are decoder-visited states less calibrated? — Ayush · W5–W7

Depends on: T1 traces (both mask types).
Why. The model was trained on random masks, but a decoder fills easy positions first and leaves unusual patterns, for example the hard positions all bunched together. If calibration breaks there, any threshold tuned on random masks is miscalibrated in exactly the states where it gets used.
How. At matched mask rates (0.1 to 0.9), compare reliability curves for uniform and decoder-induced states. Compare at the same confidence: decoder states leave harder positions, but comparing at equal ℓ removes that. Fit $a \sim f(\ell) + \text{type} + f(\ell) \times \text{type}$ and test the type terms with the document bootstrap. Repeat per decoder (top-k, threshold, margin, entropy).
What we expect. More overconfidence in decoder-induced states at the same confidence, especially at high mask rates (early steps). This is plausible, not guaranteed.
Done when. Side-by-side reliability diagrams and a difference curve with intervals.

T8 Controls: rule out the boring explanations — Parth · W6–W7

Depends on: T5, T6.
Why. Each control answers a question a reviewer would ask.
What we run and what each tells us.

- **Precision.** Re-run 200 documents in 4-bit and compare ℓ, $\Delta(m)$ and β_z with bfloat16. *Expect:* small shifts. If they are large, it is a warning to the community about quantised audits.

- **Position (PC-Sampler).** Add absolute position, distance to the nearest unmasked token, and PC-Sampler's position-aware weight as covariates. *If the effect disappears, it is positional, not selection.*
- **Token type.** Split results by punctuation and stop-words versus content words. PC-Sampler reports a bias toward such "trivial" tokens.
- **Temperature.** Fit one temperature per mask-rate bin on the first half and recompute $\Delta(m)$. *If per-step temperature removes the gap, it becomes the simple baseline C3 must beat.*

Done when. Each control appears as one row of an ablation table (main effect with and without the control, with intervals).

T9 C2 on the synthetic JSON probe (exact numbers)

Ojas · build W3–W4, run W8 (16–22 Nov)

Depends on: T3 (for fine-tuning).

Why. Only here is the true distribution p^* known, so both terms of

$$\mathrm{KL}\Big(p^* \,\Big\|\, \prod_{i\in S} p_{\theta,i}\Big) = \underbrace{\mathrm{TC}(x_S \mid x_{\text{obs}})}_{\text{(A) dependency}} + \underbrace{\sum_{i\in S} \mathrm{KL}(p_i^* \,\|\, p_{\theta,i})}_{\text{(B) estimation}}$$

can be computed exactly, in nats. The total correlation is $\mathrm{TC} = \sum_i H(p_i^*) - H(p_S^*)$.

How to build it.

1. Take 200 SchemaStore schemas and keep simple fields (enums, short strings, numbers, booleans).
2. Give each field a fixed list of values, chosen so that *every value is exactly one token* in all three tokenizers. This makes p^* exact at the token level.
3. **Dependency knob** $\rho \in \{0, 0.25, 0.5, 0.75, 1\}$: group fields into pairs. With probability ρ a field's value is determined by its partner (for example, city determines country); otherwise it is drawn independently. At $\rho = 0$, term (A) is zero.
4. Generate 1,000 evaluation instances (200 per ρ) and 200k training instances. Fine-tune the 110M model on the training set.
5. Compute $p^*(x_S \mid x_{\text{obs}})$ by enumeration. The groups are independent, so the tables stay small.

What we run. ρ × tokens per step $k \in \{1, 2, 4, 8\}$ × selection rule (random positions vs confidence top-k), on the fine-tuned 110M model (main), the un-tuned 110M, and the 8B models (a large-(B) regime).

What we expect. (A) is zero at $k = 1$ or $\rho = 0$ and grows with both. (B) does not depend on ρ and shrinks with fine-tuning. With confidence selection, (B) at committed positions shows whether selection lands on under-estimated errors. The key figure is the *crossover*: at which k and ρ dependency starts to dominate estimation.

Done when. (A)+(B) matches the directly computed KL to 10^{-6} nats (the unit test), and the crossover plot is done.

T10 C2 on real text (honest proxies)

Ojas · W8

Depends on: T5 traces, T9.

Why. To show that the probe's story holds on real text, while being clear about what can and cannot be measured.

How. On real text the true entropy is unknown, so we report proxies. The proxy for (A) is *parallel–sequential disagreement*: commit a set of k positions in parallel (one pass) and one by one (k passes), and count how often the tokens differ. Run this on 300 documents × 10 states. The proxy for (B) is the committed-token gap Δ and the cross-entropy of committed tokens. We calibrate the disagreement proxy against the true (A) on the probe and report the rank correlation.

What we expect. Disagreement grows with k, while the estimation-side gap is present even at $k = 1$. We report which side dominates at typical speeds (2–8 tokens per step), and we compare conditions rather than claiming absolute levels.

T11 C3: a recalibration step with zero extra forward passes

Parth · W9 (23–29 Nov)

Depends on: T6, T8.

Why. A* reviewers want a practical payoff, not only a measurement.

What we build.

- **C3a, context-aware threshold.** Recalibrated confidence $\hat{c}_i = \hat{\pi}(\ell_i, \text{field features})$. At each step, commit every position with $\hat{c}_i \geq 1 - \alpha$, and always at least one. The threshold now means "expected error rate α" instead of a tuned number. Note that within a step all positions share the field features, so C3a changes *how many* tokens are committed, not *which*.

- **C3b, per-position features.** Add features that differ between positions: entropy, margin, the change in ℓ since the previous step (the stability signal used by KLASS and SWD), distance to the nearest unmasked token, and token frequency. Use a small logistic model or gradient-boosted trees. Now the ranking can change too.

How. Fit on the first half of FineWeb (and, separately, on GSM8K/MBPP training data), freeze, and evaluate on test sets. Transfer check: fit on FineWeb and test on GSM8K.
What we expect. C3a at least matches the Fast-dLLM threshold at the same tokens per step, because it adapts the number of commits to the field. C3b may do better. The size of the gain is unknown. A realistic hope is a small but consistent gain on the accuracy–speed curve. If there is no gain, we report that; the measurement results still stand.

T12 Decoder comparison on real tasks — Parth · W9

Depends on: T11.
What we run. Eight decoders: confidence top-k, Fast-dLLM threshold, entropy-bounded, margin, KLASS, PC-Sampler, C3a, C3b. Tasks: GSM8K (accuracy) and MBPP and HumanEval (pass@1). Generation length 256, block length 32 (the LLaDA default), and each decoder swept over its speed setting.
How. Use real free-running generation, not teacher forcing. Use official code where it exists, and check that our top-k and Fast-dLLM numbers match published ones within 1–2 points before trusting anything else. Record accuracy, average tokens committed per step, and wall-clock time. Use 3 seeds wherever sampling is random.
What we expect. Every decoder loses accuracy as it commits more tokens per step. Better-calibrated rules should lose it more slowly. The main plot is accuracy against tokens per step, with a table at 2–3 matched speeds.

T13 Ablations and a third model — Everyone · W10–W12 (30 Nov–20 Dec)

Depends on: T5–T12.
Each ablation answers one reviewer question: (1) Does it hold for other confidence statistics (margin, entropy)? (2) Which recalibration method is enough (none, one temperature, per-step temperature, histogram binning, isotonic, context-aware)? (3) How much fitting data does C3 need (100, 300, 1,000 documents)? (4) Longer outputs (512 and 1,024 tokens, 8B models only). (5) Block-wise versus whole-sequence decoding. (6) bfloat16 versus 4-bit. (7) Contaminated documents in or out, and the time-split set. (8) A third model for E1 and E2 only. (9) Domain transfer of the calibrator. **Gate G4:** all results are frozen on 20 December.

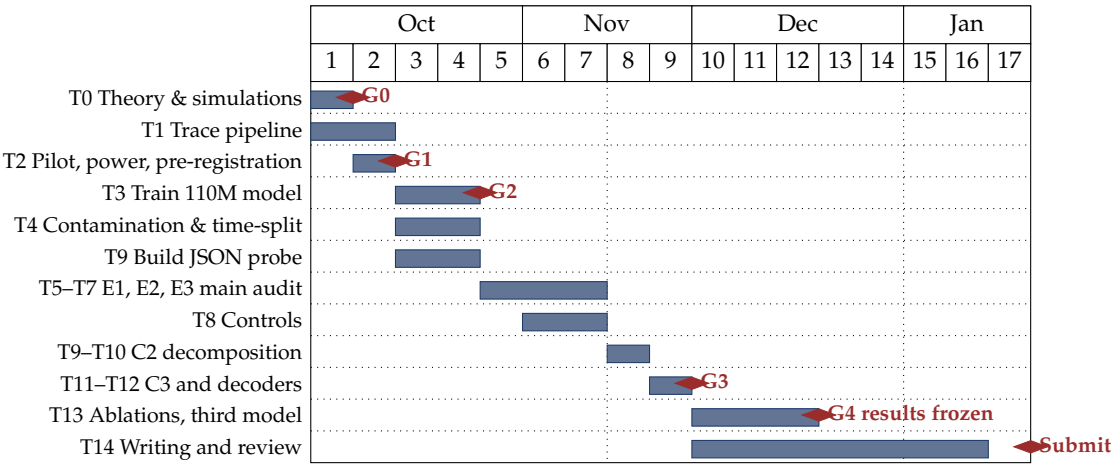
T14 Writing, internal review, and submission — Everyone (rotating lead) · draft from W2, full focus W10–W17

Depends on: everything.
The LaTeX draft lives in Overleaf from week 2. Each task owner writes their section as results arrive. Every number and figure is produced by a script from a tagged commit; nothing is typed in by hand. Draft v0 is due 6 December and v1 by 3 January, sent to 2–3 outside readers (our advisor, one diffusion-LM person, one calibration person). In week 15, each of us writes a harsh one-page mock review. Then we fix, anonymise the code, and submit at least 48 hours early.

5 Statistics rules for every task

- **The unit is the document.** Average within a document first, or resample whole documents. Pool draws and positions within a document are not independent data points.
- **Intervals.** 95% intervals come from 10,000 bootstrap resamples of documents (the percentile method).
- **Declaring an effect.** The interval must exclude zero *and* the effect must exceed twice the non-determinism floor.
- **Fit and test on different halves** of the documents, fixed by seed. Pilot documents are never reused.
- **Many models and datasets** means many tests. Apply the Holm correction across models × datasets for the pre-registered endpoints, and label everything else as exploratory.
- **Calibration metrics.** Equal-mass 15-bin ECE and smooth ECE, always with a reliability diagram. Report nulls as nulls.

6 Timeline and decision gates



Week 1 starts Monday 28 September 2026. Week 10 is 30 Nov–6 Dec, week 12 ends 20 December, and week 17 is 18–24 January.

Gate	Date	Question	If no
G0	4 Oct	Do all of us and our advisor agree with the Section 2 framing and the proofs?	Resolve before the pilot. Do not start the 8B sweep.
G1	11 Oct	Is the required $N \leq 1{,}000$ documents?	Use all 2,000 documents on fewer 8B settings, or put more weight on the 110M model and C2.
G2	25 Oct	Is the 110M model trained and sane? Does the probe identity hold?	Use the public MDLM checkpoint; fix the probe code.
G3	29 Nov	Does C3 improve the accuracy–speed curve?	Report it honestly and lead with E1, E2 and C2.
G4	20 Dec	Are all results frozen?	Cut the third model first, never the core ablations.

7 Team, rhythm, and tools

Person	Owns	Also supports
Abhiraj	T0 theory and simulations, T2 power and pre-registration, T6 field effect, statistics code	reviews all analysis code
Ayush	T1 trace pipeline, T4 contamination, T5 and T7 runs, the GPU schedule	T2 pilot runs
Ojas	T3 110M model, T9 probe, T10 real-text C2	T13 ablations
Parth	T8 controls, T11 C3, T12 decoders and tasks, the repository and results log	keeping citations checked

Rhythm. A 20-minute stand-up every Monday and Thursday, a gate review every Sunday, and a one-line entry in the shared results log for every run (date, commit, config, output path, key number). Every change to analysis code needs one reviewer. **Tools.** A GitHub repository (with separate folders for traces, analysis, and the paper), an experiment tracker (Weights & Biases or MLflow), Overleaf, and OSF for the pre-registration. **Compute (rough).** 110M training: 1–2 A100-days. Traces: a few tens of GPU-hours per 8B model. Downstream decoding: 50–80 GPU-hours per 8B model. Total: roughly 250–400 A100-hours, plus a 30% buffer.

8 The paper we are writing

Section	Pages	Main content
1 Introduction	1.0	Winner's curse intuition, teaser figure ($\Delta(m)$ vs ECE), contributions
2 Setup and theory	1.0	Eq. (1), Propositions 1–2, the field term
3 Measurement instrument	1.0	Teacher-forced infilling, the two mask types, the trace
4 Results: E1, E2, E3	1.75	Gap figure, field-effect table (β_z with and without fixed effects), trajectory shift
5 Two sources of error (C2)	1.25	Probe crossover plot, real-text proxies
6 A free fix (C3)	1.0	Accuracy vs tokens-per-step curves, matched-speed table
7 Related work, 8 Limitations	1.0	PC-Sampler, SWD/KLASS, dependency-aware decoding, learned policies; limits of the proxies
Appendix	—	Proofs, the simulation table, the full trace schema, every ablation, compute statement

9 Risks and what we do about them

Risk	What we do
The gap is tiny in 8B base models	Lead with Instruct models, decoder-induced states, and C2; report the base-model result as "why thresholding works".
The field term is zero in real models	This is still clean: C3 becomes per-step recalibration, and the paper's message is "calibrate per step, not globally".
The 110M model underfits	Use the public MDLM checkpoint for levels; keep our model for the probe.
Not enough GPU time	Cut the third model and the 1,024-token ablation first; the core audit is only a few tens of GPU-hours.
C3 gives no gain	Report the null; the paper stands on E1, E2 and C2 plus the released traces.
Someone publishes a similar idea	Post to arXiv right after submission, and release the traces as a lasting resource.
A 2026 citation is wrong or withdrawn	Parth checks every reference against arXiv or the venue before the draft freeze.

10 Final checklist before we submit

☐ Page-1 figure: $\Delta(m)$ against ECE for three models
☐ Field-effect table with and without fixed effects
☐ Main runs in bfloat16; 4-bit only as an ablation
☐ Every result on the evaluation half, with document-bootstrap intervals
☐ Contamination and time-split results in the appendix
☐ Code, traces and seeds released (anonymised for review)
☐ One-paragraph contrast each with PC-Sampler, SWD/KLASS and Jazbec et al.
☐ Propositions 1–2 with proofs checked by two people
☐ A C3 accuracy–speed plot (positive or honest null)
☐ Pre-registration link and a list of any deviations
☐ Holm correction on primary endpoints; nulls reported
☐ Every citation verified; core calibration papers cited (e.g., Guo et al., 2017)
☐ Compute statement, limitations section, venue checklist
☐ Mock reviews addressed; submitted 48 h early